

Rapport Hebdomadaire - Semaine 1

Cette semaine, nous avons fait une première amélioration du code source initial en nous concentrant sur la correction de quelques première bugs et la modularisation du code.

Correction avertissement

Tout d'abord, il est important que le code ne provoque aucun avertissement par le compilateur. Cela permet une mesure d'hygiène de code pour garantir la qualité et la sécurité avant même son exécution, voire une détection précoce des bugs critiques.

L'utilisation du flag `-Wall` avec GCC a détecté l'unique warning suivant :

```
warning: variable 'newDictionaryPrevious' set but not used [-Wunused-but-set-variable]
```

Une variable déclarée et allouée mais jamais utilisée :

Nous avons remarqué que cette variable n'a pas été oubliée d'être utilisée, mais plutôt qu'elle ne sert à rien dans l'algorithme d'insertion et peut être supprimée. C'est du **code mort** que nous devons supprimer absolument pour garantir la propreté du code.

Une fois qu'aucune erreur/avertissement n'est généré à la compilation, nous pouvons commencer à exécuter le programme et vérifier son fonctionnement.

Correction d'erreur à l'execution : Erreur de segmentation

L'execution du programme a généré une erreur de segmentation fault. Etant une erreur qui n'est souvent pas très implicite, l'utilisation de débogueur est conseillée pour tracer l'origine du problème en exécutant le programme instruction par instruction. On a utilisé GDB pour localiser l'erreur.

Avec GDB, en parcourant les instructions du code compilé avec l'option `-g`, on remarque que l'erreur provient d'un appel à `feof()` situé dans la fonction `displayDico` :

```
f = fopen(DICORES, "rw+");
```

Le mode d'ouverture `rw+` n'est pas valide pour la fonction `fopen()`. Ainsi `f` sera `NULL` et l'appel à `feof(f)` génèrera une erreur de segmentation fault. Il faudrait remplacer le mode d'ouverture par `w+`.

Il s'agit d'une erreur où le programmeur ne connaît pas parfaitement les modes d'ouverture des fichiers en C.

Il y a aussi d'autres erreurs qui ont provoqué des erreurs de segmentation fault :

Le fait que la vérification de l'ouverture du fichier n'est jamais faite pour `DICORES` et `TEXTE`. Ce qui peut aussi

généraliser des erreurs de segmentation fault si les fichiers n'existent pas ou ne peuvent pas être ouverts. Ainsi la vérification de l'ouverture des fichiers devrait être faite après chaque appel à `fopen()`.

Après ces corrections, le programme ne génère plus d'erreur de segmentation fault, et semble fonctionner en utilisant les fichiers d'exemple fournis. On a déduit que le programme étant momentanément fonctionnel.

Modularisation du code

Maintenant que le programme semble fonctionnel, nous avons remarquer que le code source était tout contenu dans un unique fichier `dico.c`. Pour améliorer la lisibilité et la maintenabilité du code, nous avons décidé de modulariser le code en plusieurs fichiers.

Nous avons procédé à une structuration par fonctionnalités, en effet dans le code source, on remarque qu'il y a des fonctions liées à la gestion du dictionnaire (`insertDico`, `displayDico`, etc.), à la gestion des mots (`next_word`, `compareWord`, etc.). Ces fonctionnalités doivent être séparées dans des modules distincts, car elles représentent des "responsabilités différentes".

Structure des modules

Nous avons décidé de tout d'abord renommer `dico.c` en `main.c` pour distinguer le programme principal du module `dico.c` qu'on va créer. Le programme est partagé entre 3 modules :

`dico.c` / `dico.h` :

Contient des méthodes/définitions et structures liées à la gestion du dictionnaire.

`word.c` / `word.h` :

Méthodes/définitions liées aux manipulations des mots.

`structNoeud.h` :

Toutes les structures qui désignent les attributs du nœud de l'ABR, utilisées par d'autres modules.

Pour le moment, on n'a pas touché les fonctions liées aux fichiers JSON car on n'a pas trouvé la bibliothèque de `/opt/json-c-master`.

Plus tard, chaque module peut être testé indépendamment.

Compilation séparées

Après la modularisation, nous avons créé un Makefile pour compiler chaque module séparément et lier les objets ensemble pour créer l'exécutable final sans avoir à recompiler tout le projet à chaque modification.

- Nous avons mis comme première règle du Makefile la règle `ALL` qui compile l'exécutable principal `main`.
- Nous avons ajouté une règle `clean` pour supprimer les fichiers objets générés.
- Pour la compilation, des fichiers objets `.o` sont générés pour chaque fichier source `.c` par les règles implicites de Makefile.

Nous avons prévu pour plus tard, un mode `DEBUG` (pour le debug du code en C) et un mode `FICJSON` (pour activer l'export JSON) dont nous pourrions "activer" via une définition de macro qui n'est pendant ce temps pas gérée. Cependant, le Makefile actuel permet tout de même de compiler le projet avec la définition de ces macros via des directives dans ce dernier, en les définissant sur la ligne de commande de `make`.

Cela nous permet d'avoir un premier Makefile fonctionnel, avec quelques options avancées pour la suite.

Bilan

Pour cette première amélioration du code source initial, nous avons d'abord corrigé les erreurs/avertissements (code mort) pour s'assurer d'une première qualité du code. Ensuite nous avons vérifié le fonctionnement du programme en corrigeant les erreurs d'exécution (GDB). Enfin, nous avons modularisé le code pour améliorer la lisibilité, la maintenabilité du code source et puis nous avons fait un premier Makefile fonctionnel.